

```
"""
Cognitive Mastery Assessment (IQ Testing
App)
Built with ipywidgets – designed to run in
Jupyter / Google Colab.

Author: Fatima Maqsood
"""

from IPython.display import display, HTML,
clear_output
import ipywidgets as widgets

# -----
# -----
# Styling
# -----
# -----
style = """
<style>
.header {
padding: 25px;
text-align: center;
background: linear-gradient(135deg,
#6366f1, #8b5cf6);
color: white;
border-radius: 20px;
margin: 15px 0;
box-shadow: 0 8px 16px rgba(0,0,0,0.15);
border: 2px solid rgba(255,255,255,0.1);
```

```
}

.logo {
  font-size: 3.5em;
  margin: 10px 0;
  text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
}

.question-box {
  background: rgba(255,255,255,0.95);
  padding: 25px;
  border-radius: 18px;
  box-shadow: 0 6px 12px rgba(0,0,0,0.08);
  margin: 20px auto;
  max-width: 800px;
}

.progress-bar {
  width: 80%;
  margin: 20px auto;
  background-color: #e5e7eb;
  border-radius: 10px;
  height: 20px;
}

.progress {
  height: 100%;
  background: linear-gradient(90deg,
#a78bfa, #8b5cf6);
  transition: width 0.5s ease;
}

.score-card {
  background: linear-gradient(135deg,
```

```

#818cf8, #c4b5fd);
    color: white;
    padding: 30px;
    border-radius: 20px;
    text-align: center;
    box-shadow: 0 8px 16px rgba(0,0,0,0.15);
}

.credits {
    text-align: center;
    color: #6b7280;
    margin: 20px;
    font-size: 0.9em;
}
</style>
"""

display(HTML(style))

# -----
# -----
# UI elements
# -----
# -----

header = widgets.HTML(
    '''<div class="header">
        <div class="logo">🌱⚡</div>
        <h1>Cognitive Mastery
Assessment</h1>
        <p>Test Your Mental Agility 🧠🧩</p>
    </div>'''
)

```

```

difficulty = widgets.Dropdown(
    options=['Easy', 'Medium', 'Hard'],
    value='Easy',
    description='Select Difficulty:',
    disabled=False,
    style={'description_width': 'initial'},
    layout={'width': '300px'}
)

start_button = widgets.Button(
    description="🚀 Begin Assessment",
    layout=widgets.Layout(width='300px'),
    style={'button_color': '#7c3aed',
'font_weight': 'bold'}
)

restart_button = widgets.Button(
    description="↺ Restart Test",
    layout=widgets.Layout(width='300px'),
    style={'button_color': '#10b981',
'font_weight': 'bold'},
    disabled=True
)

progress = widgets.HTML(
    '<div class="progress-bar"><div
class="progress" style="width: 0%"></div>
</div>'
)

question_container = widgets.Output()

credits = widgets.HTML(
    '<div class="credits">'

```

```

' 🌸 Developed with ❤️ by Fatima Maqsood
🦋<br>'
' ✨ Elevate Your Cognitive Skills!'
'</div>'
)

app_layout = widgets.VBox([
    header,
    widgets.VBox([difficulty],
        layout=widgets.Layout(align_items='center')
    ),
    widgets.HBox([start_button,
        restart_button],
        layout=widgets.Layout(justify_content='center')),
    progress,
    question_container,
    credits
],
    layout=widgets.Layout(align_items='center')
)

display(app_layout)

# -----
# -----
# Question bank
# Note: "answer" always exactly matches one
# string in "options" – no hidden
# checkmarks or trailing spaces, so
# comparison is a plain equality check.
# -----
# -----
questions = {

```

```

'Easy': [
  {"question": "What number continues
the sequence: 2, 4, 6, 8, ____? ",
    "options": ["10", "12", "14",
"16"], "answer": "10"},
  {"question": "Which shape is
different?  ",
    "options": [" Square", "
Rectangle", " Circle", " Triangle"],
    "answer": " Triangle"},
  {"question": "What is 5 + 7? ",
    "options": ["10", "12", "14",
"16"], "answer": "12"},
  {"question": "Which animal is a
mammal? ",
    "options": [" Shark", "
Dolphin", " Frog", " Snake"], "answer":
" Dolphin"},
  {"question": "What is the capital
of France? ",
    "options": [" Berlin", "
Paris", " Rome", " Madrid"], "answer":
" Paris"},
  {"question": "Which planet is
closest to the Sun? ",
    "options": [" Earth", "
Saturn", " Mercury", " Venus"],
    "answer": " Mercury"},
  {"question": "Largest ocean? ",
    "options": [" Atlantic", "
Indian", " Arctic", " Pacific"],
    "answer": " Pacific"},
  {"question": "Which month has 28
days?  17 ",

```

```

      "options": ["❄️ January", "🌸
February", "🌺 March", "All of them"],
      "answer": "All of them"},
      {"question": "√64?  $\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}$ ",
        "options": ["6", "8", "10", "12"],
        "answer": "8"},
      {"question": "Land of Rising Sun?
🇯🇵",
        "options": ["🇨🇳 China", "🇯🇵
Japan", "🇰🇷 Korea", "🇹🇭 Thailand"],
        "answer": "🇯🇵 Japan"},
    ],
    'Medium': [
      {"question": "7 × 8? ✖️",
        "options": ["48", "56", "64",
"72"], "answer": "56"},
      {"question": "Odd one out? 🍎",
        "options": ["🍎 Apple", "🍌
Banana", "🥕 Carrot", "🍊 Orange"],
        "answer": "🥕 Carrot"},
      {"question": "Next prime after 7?
 $\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}$ ",
        "options": ["9", "11", "13",
"15"], "answer": "11"},
      {"question": "Chemical symbol for
gold? 🦂",
        "options": ["Au", "Ag", "Fe",
"Cu"], "answer": "Au"},
      {"question": "Red Planet? 🟠",
        "options": ["🌍 Earth", "♂️ Mars",
"♃ Jupiter", "♄ Saturn"], "answer": "♂️
Mars"},
    ],
    'Hard': [

```

```

{"question": "Value of  $\pi$ ? 🍌",
 "options": ["3.14", "3.16",
"3.12", "3.18"], "answer": "3.14"},
{"question": "Theory of relativity? 🧪",
 "options": ["🍎 Newton", "🧠 Einstein", "🔭 Galileo", "⚡ Tesla"],
"answer": "🧠 Einstein"},
{"question": "Table salt formula? 🧂",
 "options": ["NaCl", "H2O", "CO2",
"C6H12O6"], "answer": "NaCl"},
{"question": "Planet with most moons? 🪐",
 "options": ["🌍 Earth", "🪐 Jupiter", "♂️ Mars", "♂️ Saturn"], "answer": "♂️ Saturn"},
{"question": "Largest planet in the solar system? 🌌",
 "options": ["🌍 Earth", "🪐 Jupiter", "♂️ Mars", "♂️ Saturn"], "answer": "🪐 Jupiter"},
{"question": "Who discovered penicillin? 💊",
 "options": ["🧪 Marie Curie", "🔬 Alexander Fleming", "🧠 Isaac Newton", "⚡ Nikola Tesla"],
"answer": "🔬 Alexander Fleming"},
{"question": "Speed of light in a vacuum? 🌟",
 "options": ["300,000 km/s", "150,000 km/s", "450,000 km/s", "600,000 km/s"],
"answer": "300,000 km/s"},

```

```

        {"question": "Atomic number of
oxygen? 🧪",
         "options": ["6", "7", "8", "9"],
         "answer": "8"},
        {"question": "First element on the
periodic table? 🧪",
         "options": ["Helium", "Hydrogen",
"Lithium", "Carbon"], "answer":
"Hydrogen"},
        {"question": "Who proposed the
heliocentric model? ☀️",
         "options": ["🧐 Galileo", "🧠
Copernicus", "🍏 Newton", "⚡ Tesla"],
         "answer": "🧠 Copernicus"},
    ]
}

```

```

# -----
# -----
# State + logic
# -----
# -----

current_question = 0
score = 0

def update_progress():
    total =
len(questions[difficulty.value])
    progress_percent = (current_question /
total) * 100 if total else 0
    progress.value = (
        f'<div class="progress-bar">'
        f'<div class="progress"

```

```

style="width: {progress_percent}%"></div>
</div>'
    )

def show_question():
    """Render the current question, or move
    to results if the quiz is done."""
    with question_container:
        clear_output()
        question_list =
questions[difficulty.value]
        if current_question <
len(question_list):
            question =
question_list[current_question]
            display(HTML(f'<div
class="question-box"><h3>
{question["question"]}</h3></div>'))
            option_buttons = []
            for option in
question["options"]:
                btn = widgets.Button(
                    description=option,

layout=widgets.Layout(width='auto'),
                        style={'button_color':
'#ede9fe', 'font_weight': 'bold'}
                )
                # default arg avoids late-
binding issues with loop variables
                btn.on_click(lambda b,
chosen=option: check_answer(chosen))
                option_buttons.append(btn)

```

```

display(widgets.VBox(option_buttons,
layout=widgets.Layout(align_items='center')
))

    else:
        show_results()

def check_answer(selected):
    global current_question, score
    question = questions[difficulty.value]
    [current_question]

    if selected == question["answer"]:
        score += 1
        display(HTML('<div
style="color:#10b981;text-
align:center;margin:15px">✓ Correct! 🎉
</div>'))
    else:
        display(HTML(
            f'<div
style="color:#ef4444;text-
align:center;margin:15px">✗ Incorrect 😞
<br>
            f'<span
style="color:#4c1d95">Correct:
{question["answer"]}</span></div>'
        ))

        current_question += 1
        update_progress()
        if current_question <
len(questions[difficulty.value]):

```

```

        show_question()
    else:
        show_results()

def show_results():
    with question_container:
        clear_output()
        total =
len(questions[difficulty.value])
        percentage = (score / total) * 100
if total else 0

        if percentage >= 95:
            remark, color, emoji = "🧠💥
GENIUS LEVEL! IQ 150+", "#f59e0b", "🎓✨"
            elif percentage >= 75:
                remark, color, emoji = "🚀
EXCELLENT! Superior Intelligence",
"#3b82f6", "🌟"
            elif percentage >= 50:
                remark, color, emoji = "📚
GOOD! Above Average", "#10b981", "👍"
            else:
                remark, color, emoji = "💪 KEEP
PRACTICING! You Can Improve", "#ef4444",
"📌"

        display(HTML(f'''
            <div class="score-card">
                <h2>Assessment Complete! 🏆
</h2>

                <div style="font-
size:2.5em;margin:20px">{score}/{total}

```

```

</div>
        <div style="font-
size:1.5em;color:{color}">{percentage:.1f}%
{emoji}</div>
        <h3 style="color:{color}">
{remark}</h3>
        <div style="margin-
top:20px">Difficulty: {difficulty.value}
⚡</div>
        </div>
        '''))

        restart_button.disabled = False
        difficulty.disabled = False # unlock
difficulty selection again

def start_test(b):
    global current_question, score
    current_question = 0 # <- fixed typo
(was "currcent_question")
    score = 0
    restart_button.disabled = True
    difficulty.disabled = True # lock
difficulty while quiz is running
    update_progress()
    show_question()

def restart_test(b):
    start_test(b)

```

```
start_button.on_click(start_test)  
restart_button.on_click(restart_test)
```